Essential Python for DA

- Object Oriented Programming
- Read and Write file
- Context Manager
- Handle Error
- Pandas & Numpy (on demand)
- Request API

```

```

OOP

```
class Dog:
    def __init__(self, name, species, age):
        self.name = name
        self.species = species
        self.age = age

    def bark(self):
        print(f"{self.name} is barking 'hong hong'!")

    def hbd(self):
        self.age += 1
```

```
## attributes
dog1 = Dog("milo", "Chihuahua", 2)
dog2 = Dog("david", "Husky", 3)
```

```
print(f"{dog1.name} is {dog1.species}")
```

```
milo is Chihuahua
```

```
print(f"{dog2.name} is {dog2.species}")
```

```
david is Husky
```

```
dog1.bark()
```

```
milo is barking 'hong hong'!
```

```
dog2.bark()
```

```
david is barking 'hong hong'!
```

```
print(dog1.age)
dog1.hbd()
print(dog1.age)
```

```
4
5
```

OOP

- attributes: name, species, age
- methods: function bark() hbd()

```
## homeworkd OOP for an ATM

## deposit, withdraw, check balance, OTP, order coffee, pay bill, internet etc.
```

```
class ATM:
    def __init__(self, username, bank, amount):
        self.username = username
        self.bank = bank
        self.amount = amount

    def __str__(self):
        return f"{self.username} uses {self.bank} and has {self.amount} THB in an account."

    def check_balance(self):
        print(f"Balance: {self.amount} THB")

    def deposit(self, saving):
        self.amount += saving
        print(f"Deposit {saving} THB successfully!")
```

```
toy = ATM("kasidis toy", "scb", 100)
```

```
toy.check_balance()
```

```
Balance: 100 THB
```

```
toy.deposit(200)
```

```
Deposit 200 THB successfully!
```

```
toy.check_balance()
```

```
Balance: 300 THB
```

```
print(toy)
```

```
kasidis toy uses scb and has 300 THB in an account.
```

```
## super class
```

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

## extension
class Employee(Person):
    def __init__(self, name, age, company):
        super().__init__(name, age)
        self.company = company

    def greeting(self):
        print("Hello World!")

```

```
person1 = Person("toy", 37)
```

```
person2 = Employee("lisa", 28, "google")
```

```
person2.greeting()
```

```
Hello World!
```

```
1
```

Try Except Block

```
try:
    result = 1/0
except:
    print("this is an error message!")
```

```
this is an error message!
```

```
!ls
```

```
hotel.csv  sample_data
```

```
!cat hotel.csv
```

```
hotel_id,name,city,star_rating,price_per_night
101,Grand Plaza,New York,5,350
102,Ocean View,Miami,4,220
103,Mountain Retreat,Denver,4,185
104,The Urban Stay,Chicago,3,140
105,Sunset Resort,Los Angeles,5,410
106,Desert Inn,Phoenix,3,95
107,Heritage House,Boston,4,210
108,Lakeside Lodge,Seattle,3,130
109,Golden Gate Suites,San Francisco,5,380
110,Central Boutique,Austin,4,195
```

```
## read file
import csv
```

```
try:
    file = open("hotel.csv", "r") ## r w a
    data = csv.reader(file)
    for row in data:
        print(row)
    file.close()
except FileNotFoundError:
    print("file not found, please check filename again.")
finally:
    print("read file successfully.")
```

```
['hotel_id', 'name', 'city', 'star_rating', 'price_per_night']
['101', 'Grand Plaza', 'New York', '5', '350']
['102', 'Ocean View', 'Miami', '4', '220']
['103', 'Mountain Retreat', 'Denver', '4', '185']
['104', 'The Urban Stay', 'Chicago', '3', '140']
['105', 'Sunset Resort', 'Los Angeles', '5', '410']
['106', 'Desert Inn', 'Phoenix', '3', '95']
['107', 'Heritage House', 'Boston', '4', '210']
['108', 'Lakeside Lodge', 'Seattle', '3', '130']
['109', 'Golden Gate Suites', 'San Francisco', '5', '380']
['110', 'Central Boutique', 'Austin', '4', '195']
read file successfully.
```

```python
# context manager
read_data = []

with open("hotel.csv", "r") as file:
    data = csv.reader(file)
    for row in data:
        read_data.append(row)
```

```python
# header
read_data[0]
```

```
['hotel_id', 'name', 'city', 'star_rating', 'price_per_night']
```

```python
# body data
read_data[1: ]
```

```
[['101', 'Grand Plaza', 'New York', '5', '350'],
 ['102', 'Ocean View', 'Miami', '4', '220'],
 ['103', 'Mountain Retreat', 'Denver', '4', '185'],
 ['104', 'The Urban Stay', 'Chicago', '3', '140'],
 ['105', 'Sunset Resort', 'Los Angeles', '5', '410'],
 ['106', 'Desert Inn', 'Phoenix', '3', '95'],
 ['107', 'Heritage House', 'Boston', '4', '210'],
 ['108', 'Lakeside Lodge', 'Seattle', '3', '130'],
 ['109', 'Golden Gate Suites', 'San Francisco', '5', '380'],
 ['110', 'Central Boutique', 'Austin', '4', '195']]
```

```python
## for loop to calculate avg. price per night
prices = []

for row in read_data[1: ]:
    prices.append(int(row[4])) ## -1 the last item

print(f"Average price per night: {sum(prices)/ len(prices)}")
```

```
Average price per night: 231.5
```

```python
## modern python
import pandas as pd

df = pd.read_csv("hotel.csv")

df.head()
```

	hotel_id	name	city	star_rating	price_per_night
0	101	Grand Plaza	New York	5	350
1	102	Ocean View	Miami	4	220
2	103	Mountain Retreat	Denver	4	185
3	104	The Urban Stay	Chicago	3	140
4	105	Sunset Resort	Los Angeles	5	410

Next steps: New interactive sheet

```python
df.price_per_night.mean()
```

```
np.float64(231.5)
```

```python
df["price_per_night"].mean()
```

```
np.float64(231.5)
```

```
1
```

Read & Write Files

```
import csv
```

```
# write file + context manager
head = ["id", "name", "city"]
body = [
    [1, "CU", "Bangkok"],
    [2, "LSE", "London"],
    [3, "Reading", "Reading"]
]

with open("school.csv", "w") as file:
    writer = csv.writer(file)
    writer.writerow(head)
    writer.writerows(body)
```

```
!ls
```

```
hotel.csv  sample_data  school.csv
```

```
!cat school.csv
```

```
id,name,city
1,CU,Bangkok
2,LSE,London
3,Reading,Reading
```

```
# modern python
import pandas as pd

df = pd.read_csv("school.csv")

df.head()
```

	id	name	city
0	1	CU	Bangkok
1	2	LSE	London
2	3	Reading	Reading

Next steps: New interactive sheet

```
# export csv file in pandas
df.to_csv("school2.csv")
```

```
!ls
```

```
hotel.csv  sample_data  school2.csv  school.csv
```

```
!cat school2.csv
```

```
,id,name,city
0,1,CU,Bangkok
1,2,LSE,London
2,3,Reading,Reading
```

```
## read data from internet
```

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/refs/heads/master/penguins.csv"
```

```
# import pandas as pd
import pandas as pd
df_penguin = pd.read_csv(url)
```

```
df_penguin.head()
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	MALE
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	FEMALE
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	FEMALE
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	FEMALE

Next steps: New interactive sheet

```
df_penguin.tail()
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
339	Gentoo	Biscoe	NaN	NaN	NaN	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0	FEMALE
341	Gentoo	Biscoe	50.4	15.7	222.0	5750.0	MALE
342	Gentoo	Biscoe	45.2	14.8	212.0	5200.0	FEMALE
343	Gentoo	Biscoe	49.9	16.1	213.0	5400.0	MALE

```
## select column
df_penguin[["species", "island", "body_mass_g"]].head(10)
```

	species	island	body_mass_g
0	Adelie	Torgersen	3750.0
1	Adelie	Torgersen	3800.0
2	Adelie	Torgersen	3250.0
3	Adelie	Torgersen	NaN
4	Adelie	Torgersen	3450.0
5	Adelie	Torgersen	3650.0
6	Adelie	Torgersen	3625.0
7	Adelie	Torgersen	4675.0
8	Adelie	Torgersen	3475.0
9	Adelie	Torgersen	4250.0

```
## filter rows
df_penguin[["species", "island", "body_mass_g"]] \
    .query("species == 'Adelie' and island == 'Dream'") \
    .head(10)
```

	species	island	body_mass_g
30	Adelie	Dream	3250.0
31	Adelie	Dream	3900.0
32	Adelie	Dream	3300.0
33	Adelie	Dream	3900.0
34	Adelie	Dream	3325.0
35	Adelie	Dream	4150.0
36	Adelie	Dream	3950.0
37	Adelie	Dream	3550.0
38	Adelie	Dream	3300.0
39	Adelie	Dream	4650.0

```
## R vs. Python
# df %>%
#     select() %>%
#     filter() %>%
#     arrange()

# df.select()\
#     .filter()\
#     .arrange()
```

```
## filter without query()
df_penguin[df_penguin['body_mass_g'] >= 4500][["species", "sex", "body_mass_g"]]
```

	species	sex	body_mass_g
7	Adelie	MALE	4675.0
17	Adelie	MALE	4500.0
39	Adelie	MALE	4650.0
45	Adelie	MALE	4600.0
81	Adelie	MALE	4700.0
...	...	...	...
338	Gentoo	FEMALE	4925.0
340	Gentoo	FEMALE	4850.0
341	Gentoo	MALE	5750.0
342	Gentoo	FEMALE	5200.0
343	Gentoo	MALE	5400.0

118 rows × 3 columns

```
## requests api
```

```
1
```

Requests

How to get data via API the easy way

```python
import requests

## api endpoint
url = "https://swapi.info/api/people/1"
```

```python
response = requests.get(url)
```

```python
response.json()["name"]
```

```
'Luke Skywalker'
```

```python
response.json()["height"]
```

```
'172'
```

```python
response.json()["mass"]
```

```
'77'
```

```python
## for loop
import requests
import time

characters = []

for i in range(5):
    url = f"https://swapi.info/api/people/{i+1}"
    response = requests.get(url)
    response_json = response.json()
    data = [response_json["name"],
            response_json["height"],
            response_json["mass"]]
    characters.append(data)
    print(f"Success: {i+1}")
    time.sleep(1)

print(characters)
```

```
Success: 1
Success: 2
Success: 3
Success: 4
Success: 5
[['Luke Skywalker', '172', '77'], ['C-3PO', '167', '75'], ['R2-D2', '96', '32'], ['Darth Vader', '202', '136'],
```

```python
import pandas as pd

starwars = pd.DataFrame(characters,  columns = ["name", "height", "mass"])
```

```python
starwars
```

	name	height	mass
0	Luke Skywalker	172	77
1	C-3PO	167	75
2	R2-D2	96	32
3	Darth Vader	202	136
4	Leia Organa	150	49

Next steps: New interactive sheet

1